

神奇的递归

北京理工大学计算机学院
金旭亮



以“鱼”为笔绘制“鱼”

递归的编程实现

一个构成递归调用的函数

```
void DonotRunMe() {  
    DonotRunMe();  
}
```



简言之：递归就是“自己调用自己”。

递归的定义



An algorithm is called recursive if it solves a problem by reducing it to an instance of the same problem with smaller input.

一个递归的算法，会将同一个“大”问题的“规模”不断压缩，直到易于处理为止。往往体现为代码要处理的数据量在递归前后不断“递减”。

通过实例理解递归

编个程序求n!

分析:

$$1! = 1$$

$$2! = 1*2 = 1!*2$$

$$3! = 1*2*3 = 2!*3$$

...

$$n! = 1*2*3*...*n = (n-1)!*n$$

- 从n!的数学公式可以看到，要计算n!可以先计算(n-1)!, 得到(n-1)!的结果之后，将其乘以n就得到n!。
- 这个过程可以一直持续到2!, 这时，1!=1已知，于是可以计算出2!。
- 由2!再倒推出3!, 4!, ..., 最后得到n!, 问题解决。

有了可行的算法，要计算n的阶乘，就变得很简单了：

```
//使用递归的方式计算N的阶乘
public static long calculateN(int n) {
    //数学上规定：1!=1 , 0!=1
    if (n == 1 || n == 0) {
        return 1;
    }
    //使用递归的方式计算N的阶乘
    return n * calculateN(n - 1);
}
```

CalculateN.java

什么样的问题，计算机能够解决？



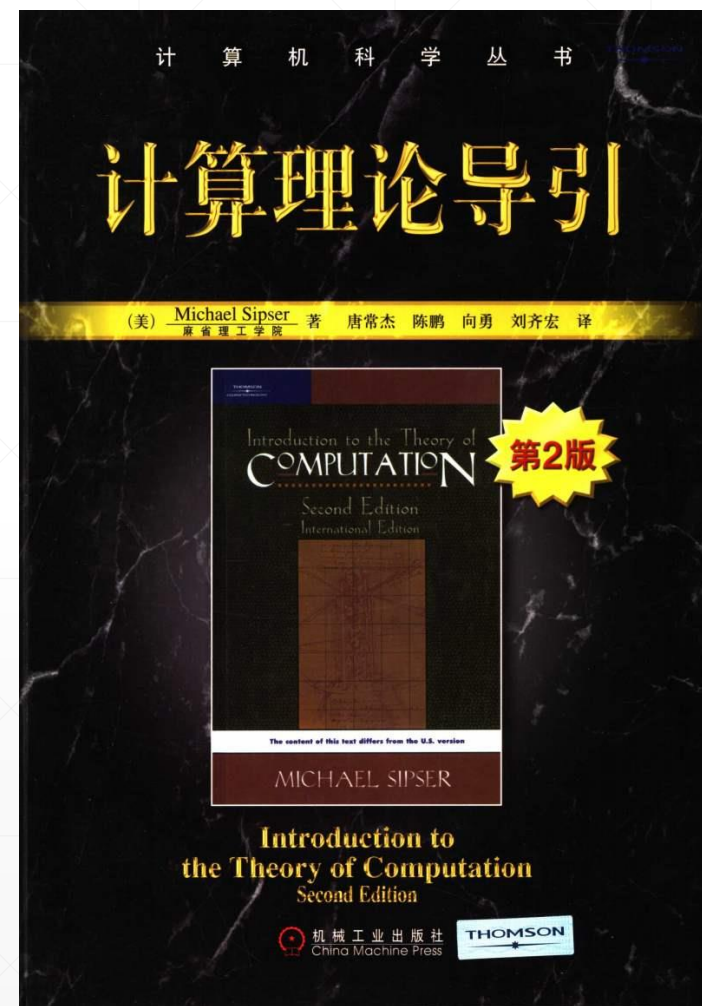
面对任何一个实际问题，如果你希望能用计算机来解决它，第一件事情就是分析清楚这个问题，看看能不能形成一个算法来解决它。



如果某问题当前还没有找到一个算法，那么，计算机就无法解决它。



研究计算机能解决的问题的理论边界在哪里，是一个很有趣的问题，感兴趣的同学可以看看《计算理论》这个领域的教材。



具体编程实现n!的递归求解

```
//使用递归的方式计算N的阶乘
public static long calculateN(int n) {
    //数学上规定: 1!=1 , 0!=1
    if (n == 1 || n == 0) {
        return 1;
    }
    //使用递归的方式计算N的阶乘
    return n * calculateN(n - 1);
}
```

■ 分析:

函数f(n): 返回n!

结束递归的条件: n=1

递归公式: $f(n)=n*f(n-1)$

- 先从大到小，再从小到大。
- 每个步骤要干的事情都是类似的，只不过其规模“小一号”。
- 必须要保证递归调用的过程可以终结。

对照实例，把握递归编程的“套路”



每个递归函数的开头一定是判断递归结束条件是否满足的语句（一般是if语句）



函数体一定至少有一句是“自己调用自己”的。



每个递归函数一定有一个控制递归可以终结的变量（通常是作为函数的参数而存在）。每次自己调用自己时，此变量会变化（一般是变小），并传递给被调用的函数。

//使用递归的方式计算N的阶乘

```
public static long calculateN(int n) {  
    //数学上规定: 1!=1 , 0!=1  
    if (n == 1 || n == 0) {  
        return 1;  
    }  
    //使用递归的方式计算N的阶乘  
    return n * calculateN(n - 1);  
}
```


用递推法求 $n!$



使用“递推”方式编程求 $n!$ ，就是使用传统的for循环的方法求 $n!$ ，让循环变量从1开始顺序递增到 n ，在循环体中累计求积。

这个工作应该很容易实现，留给同学们当作练习。

辨析：递归 vs. 递推



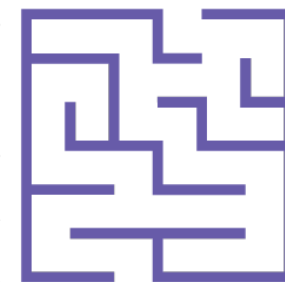
▪ 两者的区别：

1. “递归”是“由后至前再倒回来”，要求第 n 项，先求第 $n-1$ 项，……，倒推到前面的可计算出的某项，然后再返回。
2. “递推”是“从前到后”，先求第1项，然后，在此基础上求第2项，第3项，直至第 n 项，通常可以直接使用循环语句实现。



在开发中，很多算法既可以使用递归（recursive）也可以使用递推（iterative）实现，要依据具体情况进行决断。

“递归”很重要！



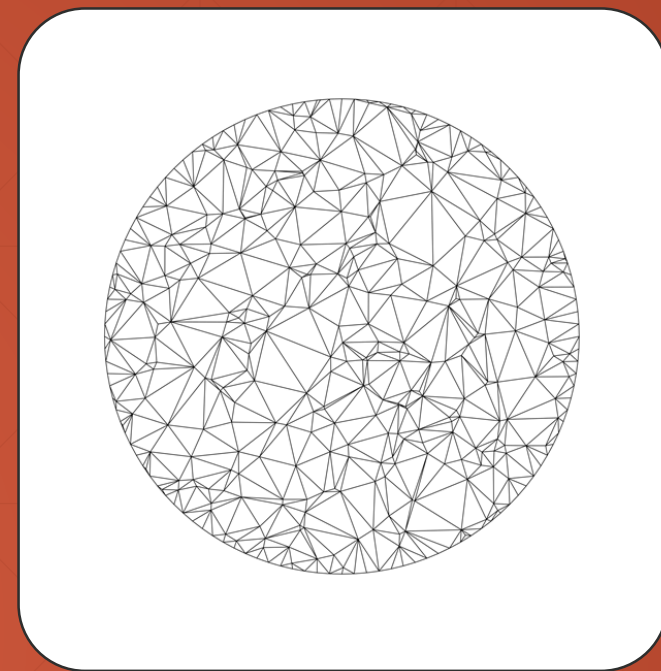
在软件科学中，递归这种思想有着重要的应用，比如，许多计算机算法都是用递归实现的。



在具体的软件开发实践中，递归也用得非常多。



对于初学者而言，要一下子理解递归比较困难，只能在开发实践中慢慢体会，最终方能灵活地应用递归解决实际问题。



处理大数字与浮点数

程序错了吗?

CalculateN示例程序中“惊现”的BUG!



```
Run: CalculateN x
"C:\Program Files\Java\jdk-15\bin\java.exe"
请输入N: 50
50! = -3258495067890909184
Process finished with exit code 0
```

阶乘值怎么可能会是一个负数?

问题的根源

java中int类型的数值占32位，是有符号的：

正数：000.....0 ~ 011.....1 即32个全“0”到31个全“1”

负数：100.....0 ~ 111.....1 即31个全“0”到31个全“1”



$\text{Integer.MAX_VALUE} = 2^{31} - 1 = 2,147,483,647$

$\text{Integer.MIN_VALUE} = -2^{31} = -2,147,483,648$

类似地，long类型的数值占64位：

$\text{Long.MAX_VALUE} = 2^{63} - 1 = 9,223,372,036,854,775,807$

$\text{Long.MIN_VALUE} = -2^{63} = -9,223,372,036,854,775,808$

问题的根源!

由于计算机使用固定的位数来保存数值，因此，能处理的数值大小是有限的，当要处理的数值超过了这一范围时，计算机将会自动截断数值的二进制表示为它所能处理的最多位数，这将导致错误的处理结果。



$50! = 304140932017133780436126081660647688443$
 $77641568960512000000000000$

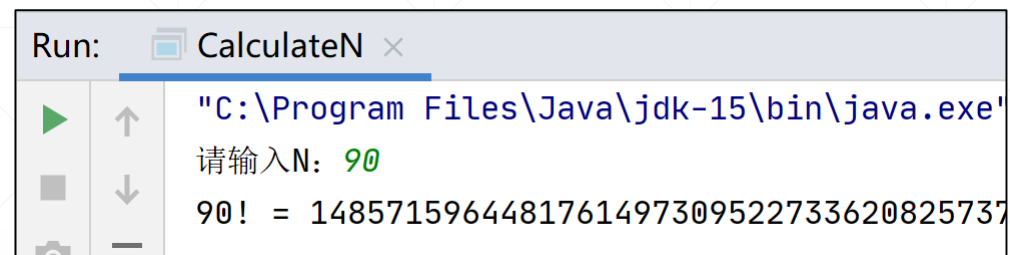
这个值实在太太，超过了long能表示的数的范围!

如何处理大的整数?



Java提供了一个BigInteger类，支持大整数的加减乘除运算。

```
//使用BigInteger解决求大数的阶乘问题
public static BigInteger calculateN2(int n) {
    if (n == 1 || n == 0) {
        return BigInteger.valueOf(1);
    }
    return BigInteger
        .valueOf(n)
        .multiply(calculateN2((n - 1)));
}
```

[illegible]



课后作业

作业₁：使用计算机计算组合数：



(1) 使用组合数公式利用 **n!** 来计算

$$C_n^k = \frac{n!}{k!(n-k)!}$$

(2) 使用 **递推** 的方法用杨辉三角形计算

$$C_{n+1}^k = C_n^{k-1} + C_n^k$$

(3) 使用 **递归** 的方法用组合数递推公式计算

背景知识：杨辉三角形与组合数公式



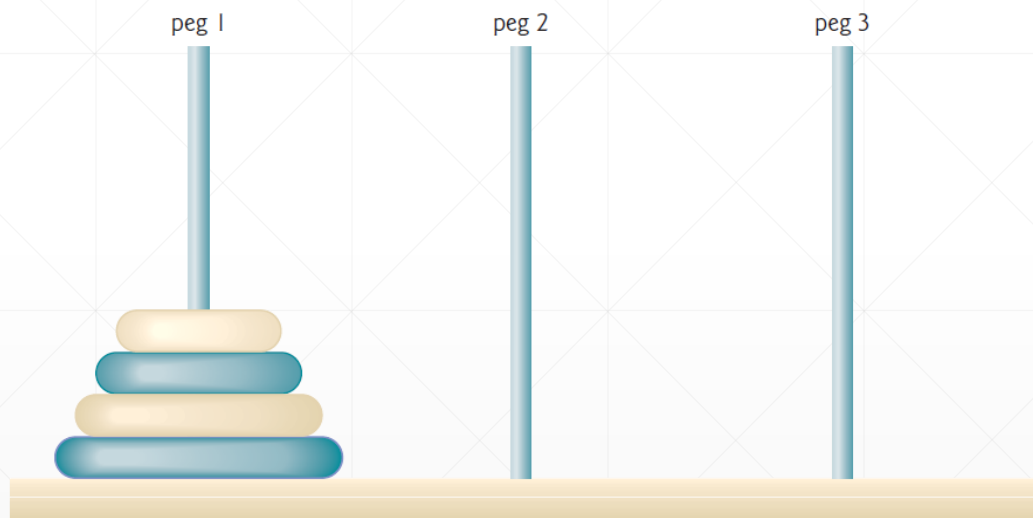
$$\begin{array}{l} \binom{0}{0} \\ \binom{1}{0} \binom{1}{1} \\ \binom{2}{0} \binom{2}{1} \binom{2}{2} \\ \binom{3}{0} \binom{3}{1} \binom{3}{2} \binom{3}{3} \end{array} \quad \begin{array}{c} 1 \\ 1 \quad 1 \\ 1 \quad 2 \quad 1 \\ 1 \quad 3 \quad 3 \quad 1 \end{array}$$

杨辉三角形可以用
来计算组合数

作业2



递归编程解决汉诺塔问题。



如果你不知道什么叫“汉诺塔问题”，请百度一下！

作业3



使用递归方式判断某个字符串是否是回文（palindrome）。

提示：

“回文”是指正着读、反着读都一样的句子。比如“我是谁是我”。

使用递归算法检测回文的算法描述如下：

A single or zero-character string is a palindrome.

Any other string is a palindrome if the first and last characters are the same, and the string that remains, excepting those characters, is a palindrome.